

Lab 3: Component Modelling & Architectural Pattern Selection

Written Justification | Problem Statement #49: Webhook Ingestion & Retry Mechanism Hub

Architecture Selection: “We chose Microservices Architecture for the Webhook Ingestion & Retry Mechanism Hub System.”

1. Architectural Choice

Style selected: Microservices. The hub is split into small, independently deployable components (Ingestion Gateway, Event Buffer, Delivery Service, Retry Manager, Dead-Letter Service, Logging Service and Management Console) that interact only through the provided and required interfaces shown in the component diagram, such as `IWebhookIngest`, `IEnqueueEvent`, `IDequeueEvent`, `IScheduleRetry`, `IDeliveryAttempt`, `IDeadLetterStore` and `ILogWrite`. *Layered* was not chosen because all layers must be scaled together and slow retries would hold shared resources. *Client-Server* was not chosen because one central server is a single point of failure, which conflicts with the 99.999% reliability target.

2. Two Reasons for the Choice

Reason 1: Independent scaling for uneven workloads. The ingestion endpoint must handle up to 2,500 requests per second, while delivery speed depends on the slowest internal target service. With microservices, the Ingestion Gateway and the Delivery Service are scaled separately, and the Event Buffer absorbs bursts between them, so extra capacity is added only where it is needed.

Reason 2: Fault isolation, so no webhook is dropped. Failing target endpoints, exponential backoff retries (1s, 2s, 4s, 8s) and dead-letter handling are confined to the Delivery Service, Retry Manager and Dead-Letter Service. If any of these slows down or fails, the Ingestion Gateway keeps accepting webhook events and the Event Buffer keeps them safe, and events that exhaust their retries are persisted in the dead-letter queue instead of being lost. Each service can also be updated and redeployed without stopping ingestion.

3. Security Advantage

Reduced attack surface through service isolation. Inbound events from third parties enter the system only through the Ingestion Gateway, so they are checked at one controlled point. The other services are reached only through their defined interfaces, and only the Delivery Service communicates with the internal target services. A fault or misuse in one service is therefore contained, and access to logs and dead-letter events is limited to the Management Console used by the Integration Developer and the System Operator.

4. Performance Benefit

Asynchronous decoupling through the Event Buffer. The Ingestion Gateway accepts a webhook event for processing and hands it to the Event Buffer without waiting for delivery to the internal service. Retry waiting time is handled by the Retry Manager instead of holding up ingestion. Together with independent scaling of the Ingestion Gateway, this keeps ingestion responsive at peak load (2,500 requests per second), even when a target service is slow or unavailable.

Trade-off Acknowledged

Microservices add network hops and operational complexity. These are limited by keeping the main event path asynchronous (Ingestion Gateway, Event Buffer, Delivery Service) and by using a single Logging Service for monitoring and audit.